Machine Translated by Google

[Chap 9] Tensorflow Basics and Regression Analysis

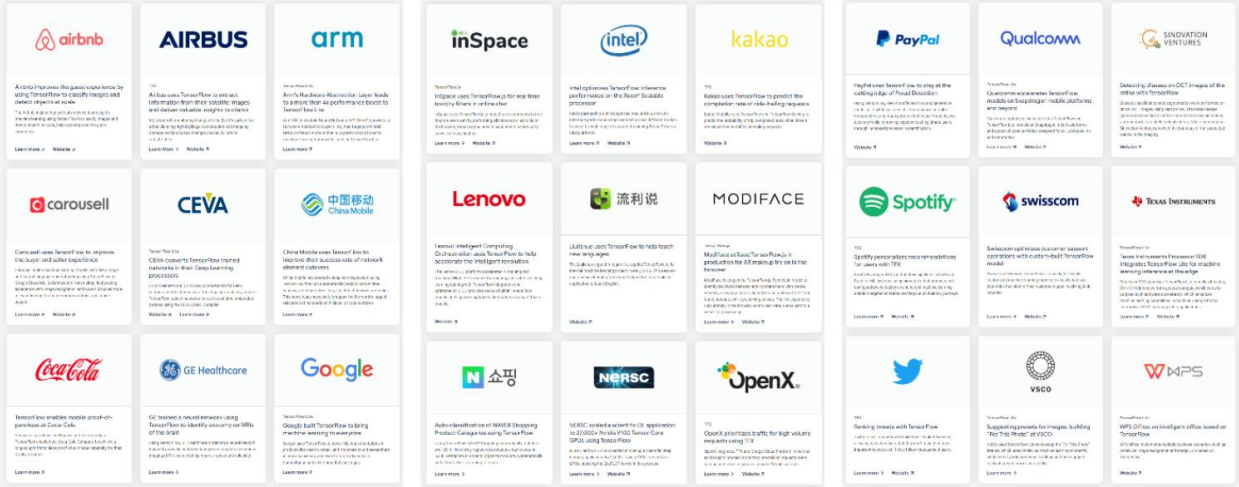
TensorFlow

Machine Translated by Google

Tensorflow

- An open-source machine learning platform distributed by Google – A set of numerical computation packages for machine learning

- Used by various companies (www.tensorflow.org)



- Integration with the Keras library for artificial neural network modeling
  - Distributed together with Google

Machine Translated by Google

Tensorflow installation

- pip install tensorflow (MacOS: pip3 install tensorflow)

```
C:\WINDOWS\system32>pip install tensorflow
Collecting tensorflow
  Downloading tensorflow-2.9.1-cp39-cp39-win_amd64.whl (444.0 MB)
     |                                        | 203.5 MB 6.4 MB/s eta 0:00:38
```

- Runs on the CPU in the default installation

  state – GPU (graphics card) utilization is required for major problems.

Machine Translated by Google

Tensor, the basic data unit of Tensorflow

- Definition of Tensor Constants

```
1  import tensorflow as tf
2  a = tf.constant(1)
3  print(a)
```

```
tf.Tensor(1, shape=(), dtype=int32)
```

– (Line 1) Load TensorFlow with the alias tf –

(Line 2) Define a tensor constant with a value of 1

- Automatically set to integer since data type was not declared
  - dtype=int32 in the output result

Machine Translated by Google

Tensor data type declaration

- Explicitly defining data types as floating-point numbers
  - It is advisable to explicitly declare data to be calculated as a floating-point number from the beginning.

```
import tensorflow as tf
a = tf.constant(1, dtype=tf.float32)
print(a)
```

```
tf.Tensor(1.0, shape=(), dtype=float32)
```

- TensorFlow Data Types (Table 9-1)

텐서플로우 자료형	의미	값의 표현 범위
tf.float32	32비트 실수	-3.4028235e+38 ~ 3.4028235e+38
tf.float64	64비트 실수	-1.7976931348623157e+308 ~ 1.7976931348623157e+308
tf.int8	8비트 정수	-128 ~ 127
tf.int16	16비트 정수	-32768 ~ 32767
tf.int32	32비트 정수	-2,147,483,648 ~ 2,147,483,647
tf.uint8	8비트 비음(nonnegative) 정수	0 ~ 255
tf.uint16	16비트 비음(nonnegative) 정수	0 ~ 65535
tf.string	문자열	모든 문자열
tf.bool	논리값 (True 또는 False)	True, False

Machine Translated by Google

Store a matrix in a Tensor

- To store a matrix, wrap the rows in [ ] and the entire matrix in [ ] again.
  - Structure similar to NumPy

- Example of a 2x2 matrix

```
import tensorflow as tf
a = tf.constant([[3.14, 1.7], [5.6, 2.2]], dtype=tf.float32)
print(a)
```

  - Output shape=(2, 2)

- Example of a 3x2 matrix

```
import tensorflow as tf
a = tf.constant([[3.14, 1.7], [5.6, 2.2], [3, 3]], dtype=tf.float32)
print(a)
```

  - Output shape=(3, 2)

Machine Translated by Google

Matrix Addition and Multiplication using TensorFlow

- Output the sum and product of two matrices

```
import tensorflow as tf
a = tf.constant([[1, 2], [3, 4]], dtype=tf.float32)
b = tf.constant([[2, 2], [3, 5]], dtype=tf.float32)
print(a+b)
print(tf.matmul(a, b))
```

```
tf.Tensor(
[[3. 4.]
 [6. 9.]], shape=(2, 2), dtype=float32)
tf.Tensor(
[[ 8. 12.]
 [18. 26.]], shape=(2, 2), dtype=float32)
```

Machine Translated by Google

TensorFlow Identity Matrix and Inverse Matrix

- Creation and operation of a 2x2 identity matrix

```
1 import tensorflow as tf
2 a = tf.constant([[1, 2], [3, 4]], dtype=tf.float32)
3 i = tf.constant(tf.eye(2), dtype=tf.float32)
4 print(tf.matmul(a, i))
5 print(tf.matmul(i, a))
```

– (Row 3) Create a 2x2 identity matrix and store it in variable i – (Rows

4–5) Output the value of a multiplied by the identity matrix: The original matrix remains unchanged

- Calculate inverse matrix

```
1 import tensorflow as tf
2 a = tf.constant([[1, 2], [3, 4]], dtype=tf.float32)
3 aInv = tf.linalg.inv(a)
4 print(aInv)
5 print(tf.matmul(a,aInv))
```

– (Line 3) Calculate the inverse of a and store it in aInv –

(Line 5) Multiplying a and aInv outputs the identity matrix

  - Slight errors may occur due to the computer's decimal limit

Machine Translated by Google

Principles of Regression Analysis

- Regression Analysis – A representative methodology for causal prediction models

- The essence of machine learning is establishing a model that explains data – a process of defining a model framework, inputting data to refine the shape of the framework to fit the data – within the framework are parameters that determine the detailed shape of the model – machine learning is finding the parameter values that best explain the data. • This process is called training, fitting, or optimization.

- Ultimately, machine learning is about optimizing the model's parameters to obtain the values predicted by the model and It can be said to be minimizing the difference between the actual correct answer values.

- Regression analysis can be understood as a type of machine learning.
  - Understanding the structure of machine learning is possible by implementing regression analysis using TensorFlow

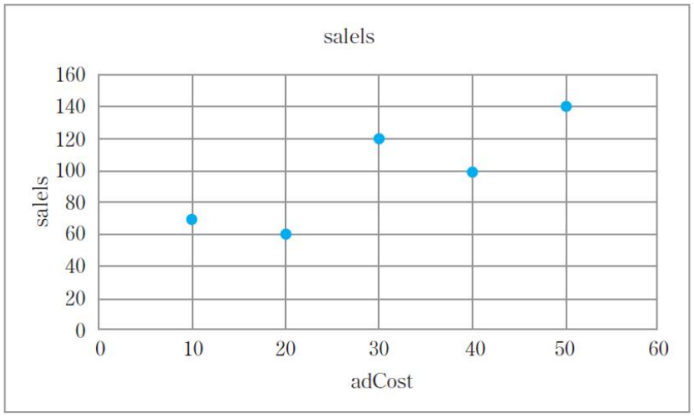
Machine Translated by Google

Regression analysis example

- Relationship between adCost and sales
  - The tendency for sales to increase as advertising costs increase

〈표 9–2〉 회귀분석 예제

adCost	sales
30	120
20	60
50	140
40	100
10	70



[그림 9–3] 광고료와 매출의 산점도

- Intending to explain this relationship within the framework of the linear equation y=ax+b
  - Let adCost be x and sales be y. – This framework has two parameters, a and b. – The objective is to optimize the values of a and b so that the difference between the calculated y and the actual sales is minimized when the ad cost is entered into x.

Machine Translated by Google

Optimization of parameters a and b

- At first, try substituting random values for a and b (a=2, b=40)
  - = 2 + 40

〈표 9–3〉 초기모델의 예측치와 실제값의 비교

x	y	$\hat{y}$	$y-\hat{y}$
30	120	100	20
20	60	80	−20
50	140	140	0
40	100	120	−20
10	70	60	10

Error

- The objective is to minimize the error (ÿÿ) – aiming to reduce this value by creating an indicator of the total error – The simple sum of errors is unsuitable as an error indicator because positive and negative values cancel each other out – Sum of Squared Errors (SSE) or Mean Squared Error (MSE) are commonly used.

- The current MSE – this $(20^2+(-20)^2+0^2+(-20)^2+10^2)/5=260$ value must be reduced by adjusting a and b

Machine Translated by Google

Optimization of parameters a and b (continued)

- Formula for calculating MSE for arbitrary a and b

$$
\begin{aligned}
MSE &= \frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2 = \frac{1}{n}\sum_{i=1}^{n}(y_i-(ax_i+b))^2 \\
&= \frac{1}{5}\left\{\begin{aligned}&(120-(30a+b))^2+(60-(20a+b))^2+(140-(50a+b))^2 \\ &+(100-(40a+b))^2+(70-(10a+b))^2\end{aligned}\right\} \qquad \text{(식 9–1)} \\
&= 1100a^2+b^2+60ab-6600a-196b+10500
\end{aligned}
$$

  – Substituting a=2 and b=40 confirms that MSE=260.

- The concept of differentiation – the instantaneous slope of a function at a specific value of a variable • Tells us how much a function increases when a variable increases very slightly • The instantaneous slope is called the gradient

- If the gradient of the MSE function with respect to a is positive, it means that increasing a increases the MSE; therefore, to reduce the MSE, a must be slightly decreased. Conversely, if the gradient is negative, a must be slightly increased to decrease the MSE.

Machine Translated by Google

Optimization of parameters a and b (continued)

Differentiating the MSE with respect to a and b yields the following:

$$\frac{\partial MSE}{\partial a} = 2200a + 60b - 6600 \qquad (식\ 9-3)$$

$$\frac{\partial MSE}{\partial b} = 60a + 2b - 196 \qquad (식\ 9-4)$$

- Substituting a=2 and b=40 into the above equation yields 200 and 4, respectively . – Since the gradient with respect to a is 200 (positive), a must be slightly decreased to reduce the MSE. – Since the gradient with respect to b is 4 (positive), b must be slightly decreased to reduce the MSE.

- As an example, let's recalculate the MSE by decreasing a and b by 0.01 each.
  – Substituting a=1.99 and b=39.99 into (Equation 9-1) yields MSE = 258.076, which is a decrease from the previous value.

- Substitute a=1.99 and b=39.99 back into (Equation 9-3) and (Equation 9-4) – The gradients of a and b are calculated as 177.4 and 3.38, respectively
  – Since both are positive, a and b must be decreased slightly further.

- By repeating this process, the optimal solution is reached when the gradient with respect to a and b approaches 0.

Machine Translated by Google

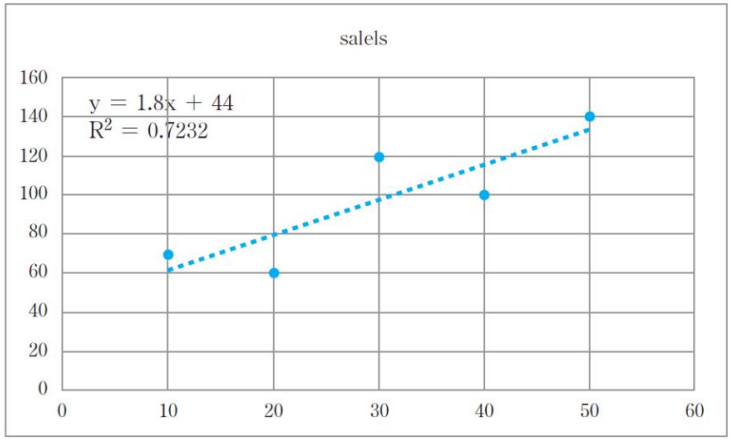
Various methods for deriving regression equations

- A method to find the mathematically optimal a and b at once
  - Set both (Equation 9-3) and (Equation 9-4) to zero and find the solution to the system of equations.

$$\begin{aligned} 2200a + 60b - 6600 &= 0 \\ 60a + 2b - 196 &= 0 \end{aligned} \qquad \text{(식 9–5)}$$

  - a=1.8, b=44 was derived

- Utilizing Excel's trendline feature
  - ^{2} = 0.7232 : This means that this model explains approximately 72% of the data variation.



[그림 9–4] 엑셀의 추세선 기능을 활용한 회귀분석

Machine Translated by Google

Various Methods for Deriving Regression Equations (Continued)

- Deriving regression equations with

  Python – Install statsmodels: pip install statsmodels

  – Perform linear regression using the OLS function

```
x = [30,20,50,40,10]
y = [120,60,140,100,70]
import statsmodels.api as sm
results = sm.OLS(y, sm.add_constant(x)).fit()
print(results.summary())
```

```
OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.723
Model:                            OLS   Adj. R-squared:                  0.631
Method:                 Least Squares   F-statistic:                     7.839
Date:                Wed, 18 May 2022   Prob (F-statistic):             0.0679
Time:                        01:47:56   Log-Likelihood:                -20.878
No. Observations:                   5   AIC:                             45.76
Df Residuals:                       3   BIC:                             44.98
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         44.0000     21.323      2.064      0.131     -23.859     111.859
x1             1.8000      0.643      2.800      0.068      -0.246       3.846
==============================================================================
```

Machine Translated by Google

Regression Analysis and Machine Learning

While the parameters of a regression model can be easily obtained through calculation or using packages, the process of gradually modifying the parameters through differentiation is similar to how machine learning works.

Machine learning consists of a repetitive process of establishing a model framework, setting parameters to initial values, identifying directions for improvement, modifying parameters, and then identifying directions for improvement again.

– I intend to implement this process using TensorFlow

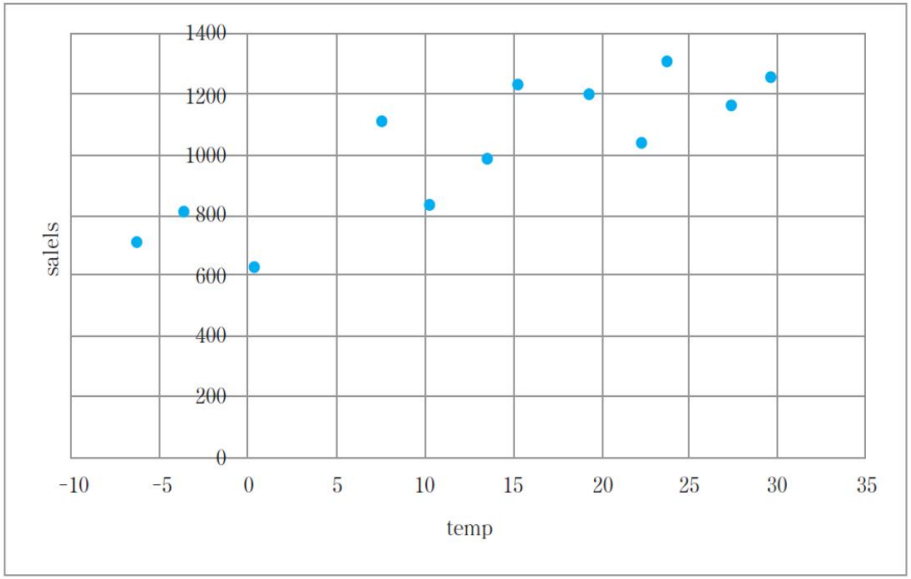
Machine Translated by Google

Example data

- softdrink.csv – The relationship between the monthly average temperature and sales revenue of a beverage company – Upward-sloping linearity: A pattern where beverage sales increase as the temperature rises – Linear regression model: Assuming the relationship y=ax+b, with temperature set as x and sales revenue as y.

〈표 9–4〉 월별 평균기온과 매출액 데이터

month (월)	temp (도)	sales (백만원)
1	-6.3	714
2	-3.6	810
3	10.2	836
4	15.3	1233
5	19.4	1204
6	22.3	1037
7	29.7	1258
8	27.3	1161
9	23.7	1310
10	13.5	986
11	7.6	1109
12	0.4	634



[그림 9–5] 기온과 매출액 관계 산점도

Machine Translated by Google

Implementing regression analysis with TensorFlow

- Read data

```
import pandas as pd
sd = pd.read_csv('softdrink.csv')
```

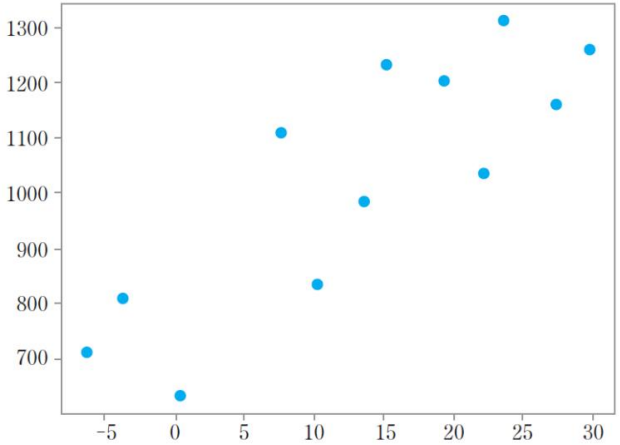
- Store data in a tensor – store

  temp (temperature) in x and sales (sales revenue) in y

```
import tensorflow as tf
x = tf.constant(sd['temp'], dtype=tf.float32)
y = tf.constant(sd['sales'], dtype=tf.float32)
print(x)
print(y)
```

- Output scatter plot

```
import matplotlib.pyplot as plt
plt.scatter(x, y, color='blue')
plt.show()
```



[그림 9-6] 기온과 매출액의 관계를 나타내는 산점도

Machine Translated by Google

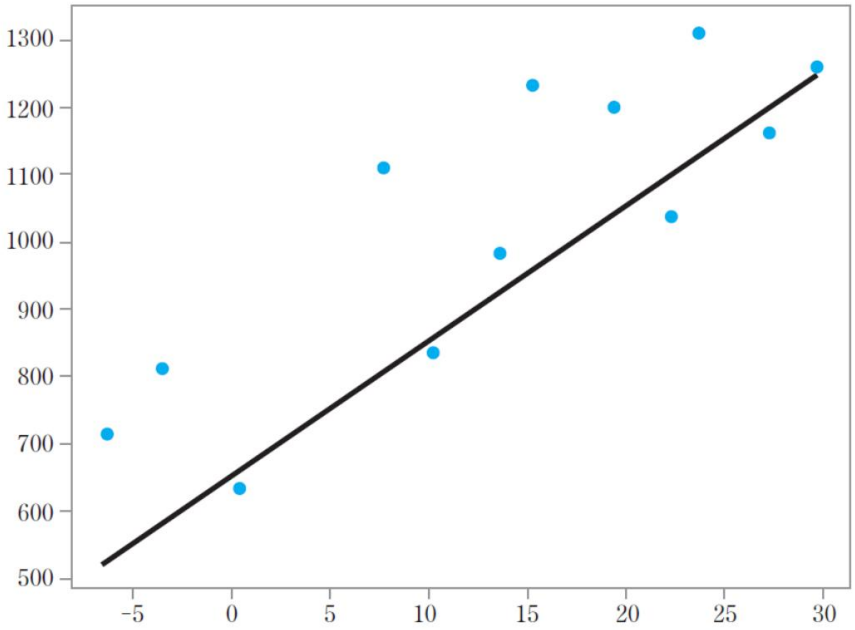
Establishing an initial model

- Set initial values for parameters a and b, calculate predicted values, and output a scatter plot.
  - (Lines 8–9) Set the initial values of a and b to 20 and 650, respectively
  - (Line 10) Calculate the predicted value y_hat using the current

  model – (Lines 12–15) Output a scatter plot of y and y_hat against x ÿ Confirm that the current model's explanatory power is poor

예제 ex9-1.py

```
1  import pandas as pd
2  sd = pd.read_csv('softdrink.csv')
3  import tensorflow as tf
4  x = tf.constant(sd['temp'], dtype=tf.float32)
5  y = tf.constant(sd['sales'], dtype=tf.float32)
6  print(x)
7  print(y)
8  a = tf.Variable(20, dtype=tf.float32)
9  b = tf.Variable(650, dtype=tf.float32)
10 y_hat = a*x+b
11 print(y_hat)
12 import matplotlib.pyplot as plt
13 plt.scatter(x, y, color='blue')
14 plt.plot(x, y_hat, color='red')
15 plt.show()
```



[그림 9-7] 초기 예측모델

Machine Translated by Google

Error measurement

- Define the calculation of the predicted value y_hat as a function for scalability

```
def y_hat(x, a, b):
    return a*x + b
```

- Output the initial model's predictions using the y_hat function

```
import pandas as pd
import tensorflow as tf

def y_hat(x, a, b):
    return a*x + b

sd = pd.read_csv('softdrink.csv')
x = tf.constant(sd['temp'], dtype=tf.float32)
y = tf.constant(sd['sales'], dtype=tf.float32)
a = tf.Variable(20, dtype=tf.float32)
b = tf.Variable(650, dtype=tf.float32)
print(y_hat(x,a,b))
```

```
tf.Tensor([ 524.   578.   854.   956. 1038. 1096. 1244. 1196. 1124.   920.   802.   658.],
shape=(12,), dtype=float32)
```

Machine Translated by Google

Error Measurement (Continued)

- Define a loss function that calculates the Mean Squared Error (MSE) between the predicted value and the actual value.

```
def loss(a, b, x, y):
    return tf.reduce_mean(tf.square(y - y_hat(x, a, b)))
```

  – tf.reduce_mean : A TensorFlow function that calculates the average of the individual values in a vector

- Calculation of error in initial model predictions

  – MSE calculated as 27767.666

  – Errors must be reduced (optimization)

예제 ex9-2.py

```
...
sd = pd.read_csv('softdrink.csv')
x = tf.constant(sd['temp'], dtype=tf.float32)
y = tf.constant(sd['sales'], dtype=tf.float32)
a = tf.Variable(20, dtype=tf.float32)
b = tf.Variable(650, dtype=tf.float32)

print('MSE = ', loss(a,b,x,y))
```

```
MSE =  tf.Tensor(27767.666, shape=(), dtype=float32)
```

Machine Translated by Google

Gradient calculation

- Calculate the gradient values for a and b respectively to reduce error – if the sign of the gradient is positive, the parameter must be slightly decreased, and if it is negative, it must be slightly increased.

- TensorFlow differentiation calculation
  - Use GradientTape to remember the function to be differentiated and the tensor variables
    - Used with Python's with.. as. syntax –

  Calculate derivatives with the gradient function

- Calculate gradient for parameters a and b

```
18 with tf.GradientTape() as t:
19     current_loss=loss(a, b, x, y)
20
21 da, db = t.gradient(current_loss, [a,b])
22 print('da = ', da)
23 print('db = ', db)
```

```
MSE =  tf.Tensor(27767.666, shape=(), dtype=float32)
da =  tf.Tensor(-1835.0167, shape=(), dtype=float32)
db =  tf.Tensor(-217.00002, shape=(), dtype=float32)
```

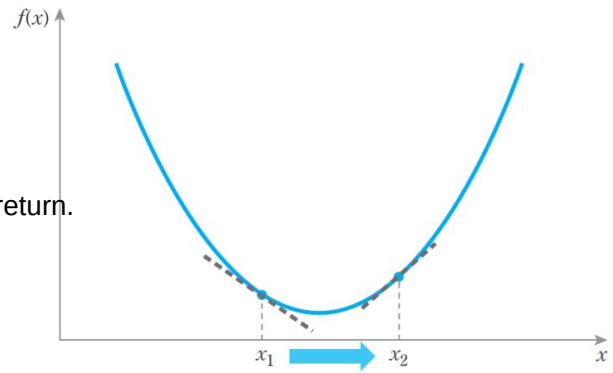
– (Lines 18–19) Record the operation of the function to be differentiated in GradientTape – (Line 21) Store da = gradient with respect to a, db = gradient with respect to b – Both da and db appear as negative numbers

Machine Translated by Google

Model Optimization: Gradient Descent

Since both da and db values are negative, a and b must be increased to decrease the loss function.

- Changing values significantly at once is risky – Since the gradient is an instantaneous slope, there is no guarantee that the slope will remain the same at the changed values if the values change drastically – In Figure 9-8, if you go from x1 to x2 all at once, the function value actually increases and the slope reverses, resulting in a situation where you must return.



[그림 9-8] 순간기울기와 학습률

- Learning rate – The degree to which a parameter changes at once – Changes the value by the amount of the learning rate multiplied by the gradient value
  - Example) Learning rate 0.001: Increase or decrease by 0.1% of the gradient at a time

- Set the learning rate to 0.001 and improve a and b

```
a.assign_add(-0.001*da)
b.assign_add(-0.001*db)
```

– Repeat this process until the gradient value becomes 0. –

Since it may not become exactly 0 due to decimal errors, terminate when the gradient is below a certain value.

Gradient Descent Method: An optimization technique that reduces loss by gradually changing the x-value based on the sign of the gradient.

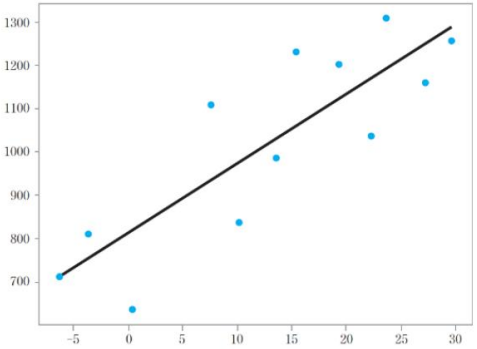
Machine Translated by Google

Regression analysis results using TensorFlow

- Implement Gradient Descent with TensorFlow
  - (Lines 17–18) Calculate the gradient of the loss function for a and b and store it in da and db
  - (Lines 23–24) Improve a and b by applying a learning rate of 0.001 – (Line 26) Determine if the optimal solution has been reached if both da and db are less than 0.05 in absolute value, terminate the while loop – (Lines 28–31) Output a scatter plot of the correct answer y for x and the final model's predicted value y_hat
- Output result
  - MSE decreased to 13896.519
    - Final calculations were a=16.00, b=811.62
  - The exact a and b using the package are, respectively Produced nearly identical values of 15.999 and 811.68

예제 ex9-4.py

```
...
16 while True:
17     with tf.GradientTape() as t:
18         current_loss=loss(a, b, x, y)
19     da, db = t.gradient(current_loss, [a,b])
20     print('MSE, a, b, da, db = ',
21         a.numpy(), b.numpy(), loss(a,b,x,y).numpy(), da.numpy(), db.numpy())
22
23     a.assign_add(-0.001*da)
24     b.assign_add(-0.001*db)
25
26     if abs(da)<0.05 and abs(db)<0.05: break
27
28 import matplotlib.pyplot as plt
29 plt.scatter(x, y, color='blue')
30 plt.plot(x, y_hat(x,a,b), color='red')
31 plt.show()
```



[그림 9-9] 최종 모델의 시각화

Machine Translated by Google

Machine Learning and Non-Linear Optimization

- MSE was used as the error function in the regression analysis.
  - – MSE is a nonlinear function containing a quadratic term – This function was optimized using gradient descent.

- Nonlinear Optimization: The Foundation of Machine Learning – Gradient Descent is one of the nonlinear optimization methods

- Importance of the learning rate – The learning rate needs to be adjusted according to the characteristics of the problem – Flexible approach, such as starting fast and slowing it down near the optimal point
  - It is also adjusted